Chapter 11: Inheritance and Polymorphism

COMP 110
Worked classes, discussion, and quizzes

Learning goals

Model an appropriate superclass and subclasses
Trace construction and overridden method calls
Distinguish overloading, overriding, and hiding
Use safe casts and define meaningful equality
Manage objects with ArrayList and a stack
Explain access rules, final, and generated code

Inheritance should describe an is-a relationship

A Circle is a GeometricObject.
A Rectangle is a GeometricObject.
Shared properties belong in the parent class.
Each subtype adds its own dimensions and behavior.

Discussion: Is a cylinder a circle, or does it have a circular base?

One shared base and two specializations

Class	Responsibility
GeometricObject	Color, filled flag, and creation timestamp
Circle	Radius, area, diameter, circumference
Rectangle	Width, height, and area

Circle and Rectangle extend GeometricObject.

A base class owns shared state

```
private String color;
private boolean filled;
private final Instant dateCreated = Instant.now();

public GeometricObject() { this("white", false); }
public GeometricObject(String color, boolean filled) {
    this.color = Objects.requireNonNull(color, "color");
    this.filled = filled;
}
```

The full file declares public class GeometricObject.

Java: GeometricObject.java

The base class exposes a small public API

```
public String getColor() { return color; }
public void setColor(String color) {
    this.color = Objects.requireNonNull(color, "color");
}
public boolean isFilled() { return filled; }
public void setFilled(boolean filled) { this.filled = filled; }
public Instant getDateCreated() { return dateCreated; }
@Override public String toString() {
    return "color=" + color + ",filled=" + filled;
}
```

The inherited methods remain available to clients of a subclass.

Java: GeometricObject.java

A Circle constructs its parent part

```
private double radius;
public Circle() { this(1); }
public Circle(double radius) { this(radius, "white", false); }
public Circle(double radius, String color, boolean filled) {
    super(color, filled);
    setRadius(radius);
}
```

super(color, filled) chooses a GeometricObject constructor.

Java: Circle.java

A subclass adds its own behavior

```
public double getRadius() { return radius; }
public void setRadius(double radius) {
    if (!Double.isFinite(radius) || radius < 0) {
        throw new IllegalArgumentException("Invalid radius");
    }
    this.radius = radius;
}
public double getArea() { return Math.PI * radius * radius; }
public double getDiameter() { return 2 * radius; }
public double getCircumference() {
    return 2 * Math.PI * radius;
}
```

this.radius = radius stores the parameter in the object field.

Java: Circle.java

A second subclass reuses the same parent

```
private final double width;
private final double height;
public Rectangle(double width, double height) {
    super();
    if (!Double.isFinite(width) || !Double.isFinite(height)
            || width < 0 || height < 0) {
        throw new IllegalArgumentException(
    "Invalid dimensions");
    }
    this.width = width;
    this.height = height;
}
```

Rectangle has dimensions in addition to inherited shared properties.

Java: Rectangle.java

A client uses inherited and added methods

```
Circle circle = new Circle(2, "red", true);
Rectangle rectangle = new Rectangle(3, 4);
System.out.println(circle.getColor());
System.out.println(circle.isFilled());
System.out.printf(Locale.US, "Circle area: %.2f%n",
    circle.getArea());
System.out.println("Rectangle area: " + rectangle.getArea());
System.out.println(circle.getDateCreated() != null);
```

The same Circle object has shared state and radius-specific behavior.

Java: TestCircleRectangle.java

Trace a chain of constructors

```
static class Person {
    Person() { System.out.println("Person"); }
}
static class Employee extends Person {
    Employee() { this("employee");
    System.out.println("Employee no-arg"); }
    Employee(String role) { super();
    System.out.println("Employee arg"); }
}
static class Faculty extends Employee {
    Faculty() { super(); System.out.println("Faculty"); }
}
```

The driver constructs one Faculty object.

Java: ConstructorOrderDemo.java

Constructor output follows the returning call chain

Completing constructor	Output
Person()	Person
Employee(String)	Employee arg
Employee()	Employee no-arg
Faculty()	Faculty

There is one Faculty object whose initialization includes its parent parts.

Java: ConstructorOrderDemo.java

A parent without a no-arg constructor

```
private final String name;
public RequiredFruit(String name) { this.name = name; }
public String getName() { return name; }
```

RequiredFruit defines only a constructor that takes a name.

Java: RequiredFruit.java

The child must choose an available parent constructor

```
public Apple() { super("apple"); }
```

The declaration is public class Apple extends RequiredFruit.

Java: Apple.java

A constructor repair that compiles

```
Apple apple = new Apple();
System.out.println(apple.getName());
```

getName is inherited from RequiredFruit.

Java: FruitConstructorDemo.java

Quiz 1: constructor chains

1. Are constructors inherited?
2. What happens if a missing super() target is implied?
3. In Java 17, where does super(...) go?
4. Does super(...) allocate a second object?
5. In the trace, which message prints first?

Call the inherited implementation with super

```
@Override public String toString() {
    return super.toString() + ",radius=" + radius;
}
```

The override extends the parent text instead of calling itself.

Java: Circle.java

Repair the source example by clarifying the model

```
private final Circle base;
private final double height;
public Cylinder(double radius, double height) {
    if (!Double.isFinite(height) || height < 0) {
        throw new IllegalArgumentException("Invalid height");
    }
    this.base = new Circle(radius);
    this.height = height;
}
public double getVolume() { return base.getArea() * height; }
```

A cylinder has a Circle base; volume has its own method name.

Java: Cylinder.java

Discussion: reuse without an incorrect is-a claim

```
Cylinder cylinder = new Cylinder(2, 3);
System.out.printf(Locale.US, "Volume: %.2f%n",
    cylinder.getVolume());
```

Which design better communicates the dimensions and units?

Java: CylinderDemo.java

Overriding and overloading solve different problems

```
static class Parent {
    public String show(Object value) { return "Parent Object"; }
}
static class Child extends Parent {
    @Override public String show(Object value) {
    return "Child Object";
}
    public String show(String value) { return "Child String"; }
}
```

The Object signature is overridden. The String signature is an overload.

Java: MethodMatchingDemo.java

Trace matching first, then binding

```
Parent ref = new Child();
Child child = new Child();
Object argument = "hi";
System.out.println(ref.show("hi"));
System.out.println(child.show("hi"));
System.out.println(child.show(argument));
```

Predict Child Object, Child String, Child Object.

Java: MethodMatchingDemo.java

An overriding reference return can be more specific

```
static class Parent {
    public Number getNumber() { return 1; }
}
static class Child extends Parent {
    @Override public Integer getNumber() { return 2; }
}
```

Integer is a subtype of Number, so this covariant return is valid.

Java: CovariantReturnDemo.java

Static methods are hidden, not overridden

```
Parent ref = new Child();
System.out.println(Parent.kind());
System.out.println(Child.kind());
System.out.println(ref.kind()); // deliberate reading exercise
System.out.println(ref.describe());
```

Use class names for static calls in normal code.

Java: StaticHidingDemo.java

A same-name private method does not override

```
static class Parent {
    private String getInfo() { return "Parent"; }
    public void printPerson() { System.out.println(getInfo()); }
}
static class Child extends Parent {
    private String getInfo() { return "Child"; }
}
```

Both driver calls print Parent.

Java: PrivateMethodsDemo.java

Quiz 2: method behavior

1. Does a different parameter list create an override?
2. Can static methods be overridden?
3. Can a private parent method be overridden?
4. Can an override return Integer for a Number return?
5. What does @Override ask the compiler to check?

Object supplies a default toString

```
Object value = new Object();
System.out.println(value.toString());
```

Its default text is a class name, @, and a hexadecimal hash-code value.

Java: ObjectToStringDemo.java

A parent method can call an overridable method

```
private final String name;
public Person(String name) { this.name = name; }
public String getName() { return name; }
public String getInfo() { return "Person"; }
public void printPerson() { System.out.println(getInfo()); }
@Override public String toString() { return "Person:" + name; }
```

printPerson calls getInfo on the current receiver.

Java: Person.java

A subclass can specialize inherited behavior

```
public Student(String name) { super(name); }
@Override public String getInfo() { return "Student"; }
@Override public String toString() {
    return "Student:" + getName();
}
```

The Student overrides return student-specific text.

Java: Student.java

A more specific class can inherit an override

```
public GraduateStudent(String name) { super(name); }
```

GraduateStudent inherits Student.getInfo and Student.toString.

Java: GraduateStudent.java

The inherited caller still dispatches on its receiver

```
new Person("Ada").printPerson();
new Student("Bo").printPerson();
new GraduateStudent("Cy").printPerson();
```

The results are Person, Student, Student.

Java: DynamicBindingDemo.java

One Object parameter accepts several reference types

```
public static void show(Object value) {
    System.out.println(value.toString());
}

public static void main(String[] args) {
    show(new Person("Ada"));
    show(new Student("Bo"));
    show(new GraduateStudent("Cy"));
    show(new Circle(2));
}
```

The chosen toString implementation follows the actual object.

Java: PolymorphismDemo.java

An int array is an Object, but not an Object array

```
Integer[] integers = {12, 24};
Double[] decimals = {12.4, 24.0};
int[] primitives = {1, 2, 3};
printArray(integers);
printArray(decimals);
System.out.println(Arrays.toString(primitives));
// printArray(primitives); // int[] is not an Object[]
```

Integer[] and Double[] can be passed to printArray(Object[]).

Java: BoxedArrayDemo.java

Quiz 3: declared type and runtime type

1. What can an Object variable call without a cast?
2. Which toString applies to a GraduateStudent here?
3. Does an int[] convert to Object[] by boxing?
4. Are overloads selected from runtime argument types?
5. Does Object.toString specify a memory address?

A cast changes the reference view, not the object

```
Object value = new Circle(2);
System.out.println(value.toString());
// value.getRadius(); // not declared in Object
Circle circle = (Circle) value;
System.out.println(circle.getRadius());
```

The object is still the same Circle before and after the cast.

Java: CompileTypeDemo.java

Guard a downcast with instanceof

```
public static void display(GeometricObject value) {
    if (value instanceof Circle) {
        Circle circle = (Circle) value;
        System.out.printf(Locale.US,
    "Circle: %.2f, diameter %.1f%n",
            circle.getArea(), circle.getDiameter());
    } else if (value instanceof Rectangle) {
        Rectangle rectangle = (Rectangle) value;
        System.out.println("Rectangle: " + rectangle.getArea());
    }
}
```

The base reference supports shared methods; the cast exposes subtype methods.

Java: CastingDemo.java

Java 16+: bind a variable in the successful type test

```
Object value = new Circle(2);
if (value instanceof Circle circle) {
    System.out.println(circle.getRadius());
} else if (value instanceof Rectangle rectangle) {
    System.out.println(rectangle.getArea());
}
```

The pattern variable is available where the successful match guarantees its type.

Java: PatternMatchingDemo.java

A permitted cast can still fail at runtime

```
Object value = new Rectangle(2, 3);
try {
    Circle circle = (Circle) value;
    System.out.println(circle.getRadius());
} catch (ClassCastException ex) {
    System.out.println("ClassCastException");
}
Circle missing = (Circle) null;
System.out.println(missing == null);
System.out.println(missing instanceof Circle);
```

Casting null to a reference type gives null; instanceof on null is false.

Java: CastAndNullDemo.java

Discussion: what should a client rely on?

Which operations belong to every GeometricObject?
Which belong only to Circle or Rectangle?

Would a common getArea contract simplify the client?
When is a cast evidence that the abstraction
needs a different public operation?

Identity and value equality ask different questions

```
Circle first = new Circle(2);
Circle second = new Circle(2);
Circle alias = first;
System.out.println(first == second);
System.out.println(first.equals(second));
System.out.println(first == alias);
```

Our mutable Circle keeps Object identity equality.

Java: IdentityDemo.java

Define a narrow immutable value type

```
private final double radius;
public CircleValue(double radius) {
    if (!Double.isFinite(radius) || radius < 0) {
        throw new IllegalArgumentException("Invalid radius");
    }
    this.radius = radius == 0 ? 0 : radius;
}
public double getRadius() { return radius; }
```

The class is final and stores one validated radius.

Java: CircleValue.java

Override equals(Object) and hashCode together

```
@Override public boolean equals(Object value) {
    if (this == value) return true;
    if (!(value instanceof CircleValue)) return false;
    CircleValue other = (CircleValue) value;
    return Double.compare(radius, other.radius) == 0;
}
@Override public int hashCode() {
    return Double.hashCode(radius);
}
```

Equal values must have equal hash codes.

Java: CircleValue.java

Test the equality policy, not only one happy case

```
CircleValue first = new CircleValue(2);
CircleValue second = new CircleValue(2);
System.out.println(first == second);
System.out.println(first.equals(second));
System.out.println(first.hashCode() == second.hashCode());
System.out.println(first.equals(null));
System.out.println(new CircleValue(-0.0).equals(
    new CircleValue(0.0)));
```

Test identity, equal values, null, unequal values, and boundary values.

Java: EqualityDemo.java

equals(BadValue) is an overload, not the Object override

```
BadValue first = new BadValue(2);
BadValue second = new BadValue(2);
Object asObject = second;
System.out.println(first.equals(second));
System.out.println(first.equals(asObject));
```

The full file declares equals(BadValue), so the Object method is not overridden.

Java: EqualsOverloadDemo.java

Quiz 4: casts and equality

1. Does a cast construct a different object?
2. What is null instanceof Circle?
3. Does equals always mean same field values?
4. Why also override hashCode?
5. Is equals(CircleValue) the Object signature?

Array and ArrayList operations

Task	Array / ArrayList
Size	array.length / list.size()
Read an element	array[index] / list.get(index)
Replace an element	array[index] = value / list.set(index, value)
Change number of elements	New array needed / add and remove operations

ArrayList grows within implementation and memory limits, not without limit.

Create, insert, read, and replace

```
ArrayList<String> cities = new ArrayList<>();
cities.add("London");
cities.add("Paris");
cities.add(1, "Tokyo");
System.out.println(cities);
System.out.println(cities.get(0));
System.out.println(cities.set(1, "Oslo"));
System.out.println(cities.contains("Paris"));
System.out.println(cities.indexOf("Paris"));
```

Inserting shifts later elements. set replaces an existing element.

Java: TestArrayList.java

Remove values and clear the list

```
System.out.println(cities.remove("London"));
System.out.println(cities.remove(0));
System.out.println(cities);
cities.clear();
System.out.println(cities.size());
```

The value-removal overload returns a boolean; index removal returns the removed element.

Java: TestArrayList.java

For Integer lists, removal overloads are easy to confuse

```
ArrayList<Integer> values = new ArrayList<>(
    Arrays.asList(1, 2, 3, 2));
values.remove(2);
System.out.println(values);
values.remove(Integer.valueOf(2));
System.out.println(values);
```

remove(2) removes index 2; remove(Integer.valueOf(2)) removes a matching value.

Java: ListRemovalDemo.java

Remove by index without skipping shifted elements

```
ArrayList<Integer> values = new ArrayList<>(
    Arrays.asList(1, 2, 4, 5));
for (int i = values.size() - 1; i >= 0; i--) {
    if (values.get(i) % 2 == 0) values.remove(i);
}
System.out.println(values);
```

Walking backward keeps earlier unvisited indexes stable.

Java: ListIterationDemo.java

Collect distinct values in first-seen order

```
Scanner input = new Scanner(System.in);
System.out.println("Enter ints; 0 ends:");
ArrayList<Integer> values = new ArrayList<>();
int value = input.nextInt();
while (value != 0) {
    if (!values.contains(value)) values.add(value);
    value = input.nextInt();
}
System.out.println(values);
```

The sentinel is excluded before the list is updated.

Java: DistinctNumbers.java

A copied list has its own structure, with shared elements

```
ArrayList<Circle> original = new ArrayList<>();
original.add(new Circle(2));
ArrayList<Circle> copy = new ArrayList<>(original);
original.get(0).setRadius(3);
original.add(new Circle(4));
System.out.println(original.size() + " " + copy.size());
System.out.println(copy.get(0).getRadius());
```

The copy constructor does not clone the Circle objects.

Java: ListCopyDemo.java

Convert between an object array and a resizable list

```
String[] array = {"red", "green", "blue"};
ArrayList<String> list = new ArrayList<>(Arrays.asList(array));
list.set(0, "black");
String[] result = list.toArray(new String[0]);
System.out.println(Arrays.toString(array));
System.out.println(Arrays.toString(result));
```

The new ArrayList copies the sequence of element references.

Java: ArrayListFromArrayDemo.java

Arrays.asList alone is a fixed-size view

```
String[] array = {"red", "green"};
List<String> view = Arrays.asList(array);
view.set(0, "black");
System.out.println(Arrays.toString(array));
try { view.add("blue"); }
catch (UnsupportedOperationException ex) {
    System.out.println("Fixed-size view");
}
ArrayList<String> copy = new ArrayList<>(view);
copy.add("blue");
System.out.println(copy);
```

Use new ArrayList<>(...) when a separate resizable structure is required.

Java: ArraysAsListDemo.java

Sort and find extrema using a stated order

```
ArrayList<String> colors = new ArrayList<>(
    Arrays.asList("red", "green", "blue"));
Collections.sort(colors);
System.out.println(colors);
System.out.println(Collections.min(colors));
System.out.println(Collections.max(colors));
```

Natural String order gives blue, green, red.

Java: ListSortDemo.java

Shuffle changes order while preserving the elements

```
ArrayList<Integer> values = new ArrayList<>(
    Arrays.asList(3, 5, 95, 4, 15, 34, 3, 6, 5));
Collections.shuffle(values, new Random(7));
System.out.println(values);
```

A seed makes a classroom run repeatable under a given implementation.

Java: ListShuffleDemo.java

Quiz 5: list behavior

1. Is ArrayList capacity the same as size?
2. Does set add a new element?
3. What does remove(2) mean for ArrayList<Integer>?
4. Is new ArrayList<>(old) a deep element copy?
5. Does Arrays.asList(array) support add?

A stack exposes last-in first-out operations

push adds at the top.
peek reads the top without removing it.
pop removes and returns the top.
isEmpty and getSize describe the current state.

A stack can use an ArrayList internally.

Hide the list behind stack operations

```
private final ArrayList<Object> elements = new ArrayList<>();
public boolean isEmpty() { return elements.isEmpty(); }
public int getSize() { return elements.size(); }
public void push(Object value) {
    elements.add(Objects.requireNonNull(value, "value"));
}
```

final prevents replacing the list reference; the list contents still change.

Java: MyStack.java

Define empty-stack behavior explicitly

```
public Object peek() {
    if (isEmpty()) {
    throw new NoSuchElementException("Empty stack");
}
    return elements.get(elements.size() - 1);
}
public Object pop() {
    if (isEmpty()) {
    throw new NoSuchElementException("Empty stack");
}
    return elements.remove(elements.size() - 1);
}
```

peek and pop both target index size - 1.

Java: MyStack.java

Trace the top through push, peek, and pop

```
MyStack stack = new MyStack();
stack.push("Java");
stack.push(42);
stack.push("CS");
System.out.println(stack.getSize());
System.out.println(stack.peek());
System.out.println(stack.pop());
System.out.println(stack.peek());
System.out.println(stack);
System.out.println(stack.search(42));
```

The last pushed value is the first popped value.

Java: TestStack.java

Test the stack's rejected operations

```
MyStack stack = new MyStack();
try { stack.pop(); }
catch (NoSuchElementException ex) {
    System.out.println("Empty pop rejected");
}
try { stack.push(null); }
catch (NullPointerException ex) {
    System.out.println("Null push rejected");
}
System.out.println(stack.isEmpty());
```

An operation that fails validation leaves this stack unchanged.

Java: EmptyStackDemo.java

Access to a class member

Modifier	Access rule in this chapter
private	Within the declaring class and its nest
No modifier	Within the same package
protected	Same package, plus restricted subclass access outside it
public	Where the enclosing type and member are accessible

Protected does not mean unrestricted access through any parent reference.

Access example in a real base package

```
private String secret = "secret";
String packageOnly = "package";
protected String label = "base";
public String publicLabel = "public";
public String readSecret() { return secret; }
```

The file declares package access.base; before the class.

Java: access/base/AccessBase.java

A subclass can use protected state under its access rules

```
public String expose() { return label; }
public String allowed(AccessChild other) { return other.label; }
// Not legal here: return other.label if other is AccessBase.
// packageOnly and secret are not accessible here.
```

An AccessChild qualifier is allowed here; an arbitrary AccessBase qualifier is not.

Java: access/child/AccessChild.java

A nearby client is not automatically a subclass

```
AccessChild child = new AccessChild();
System.out.println(child.expose());
System.out.println(child.allowed(new AccessChild()));
System.out.println(child.publicLabel);
System.out.println(child.readSecret());
// child.label; // this client is not a subclass of AccessBase
```

Run this packaged driver as access.child.ProtectedAccessDemo.

Java: access/child/ProtectedAccessDemo.java

A peer in the declaring package has package access

```
AccessBase value = new AccessBase();
System.out.println(value.packageOnly);
System.out.println(value.label);
// value.secret; // private in AccessBase
```

Run this driver as access.base.PeerDemo.

Java: access/base/PeerDemo.java

An override can widen access, but cannot weaken it

```
static class Parent {
    protected String label() { return "Parent"; }
}
static class Child extends Parent {
    @Override public String label() { return "Child"; }
}
```

The public override is accessible wherever the parent protected contract was.

Java: VisibilityWidenDemo.java

final has different effects on classes, methods, and variables

```
final int limit = 3;
final ArrayList<String> names = new ArrayList<>();
names.add("Java");
System.out.println(limit);
System.out.println(names);
// names = new ArrayList<>(); // cannot reassign
// limit = 4; // cannot reassign
```

A final reference can still refer to a mutable object.

Java: FinalRulesDemo.java

Understand the public API before generating boilerplate

```
private final String name;
public PlainProfile(String name) { this.name = name; }
public String getName() { return name; }
@Override public String toString() { return "Profile:" + name; }
```

A constructor, getter, and toString are ordinary Java methods.

Java: PlainProfile.java

Optional discussion: code generation and build configuration

```
PlainProfile profile = new PlainProfile("Ada");
System.out.println(profile.getName());
System.out.println(profile);
```

What would a tool generate, and how would you verify its public behavior?

Java: BoilerplateDemo.java

Quiz 6: stack, visibility, and final

1. Does peek remove an element?
2. What is the top index in our list-backed stack?
3. Can protected be used by any client in another package?
4. Can an override reduce public to protected?
5. Does final ArrayList prevent add?

Exit exercise: reverse words with MyStack

Read whitespace-separated words until end of input.
Push each word.
Pop until the stack is empty and print each result.

For one two three, expect three two one.
Test empty input and a single word too.

Exit exercise: one possible solution

```
Scanner input = new Scanner(System.in);
MyStack stack = new MyStack();
while (input.hasNext()) stack.push(input.next());
while (!stack.isEmpty()) {
    System.out.print(stack.pop() + " ");
}
System.out.println();
```

The client relies on push, pop, and isEmpty.

Java: StackReverseWords.java

Chapter review

Use inheritance for an appropriate is-a contract.
Trace constructor calls and method dispatch separately.
Match overloads before selecting runtime overrides.
Guard downcasts and define equality deliberately.
Distinguish list size, structure, and shared elements.
Keep access and mutation aligned with the public API.